

Jiashuo Wang(364341)- DBSAssignment1

A- Explain with your own words what DISTINCT removes.

The DISTINCT keyword removes duplicate rows from the query results. It ensures that each value returned is unique. For example, even if 1000 books belong to the "currently-reading" shelf, the result will only display "currently-reading" once.

F3- Describe what a full outer join between publisher and book will reveal?

A FULL OUTER JOIN between publisher and book will reveal all matches between the two tables, as well as all unmatched records from both sides. Specifically, it will show books that have no publisher assigned to them (filling publisher columns with NULLs), and publishers that currently have zero books in our database (filling book columns with NULLs).(All in all, show all the data.)

F4- In your own words, describe how an inner join joins rows

An INNER JOIN combines rows from two tables by strictly looking for matches based on a related column (the ON condition). It acts like a strict filter: it only returns the rows where there is a match in BOTH tables. If a row in the first table cannot find a matching row in the second table, it is completely excluded from the final result.

G- Then try using both the UNION and the UNION ALL keywords on the sets. Is there a difference in the results you get? Describe why/why not.

Yes, UNION combines the two sets but automatically removes any duplicate rows. If a title appears in both shelves, it will only be listed once in the final output.

UNION ALL simply stacks the two sets together without checking for duplicates. It retains all rows, meaning a title will be listed twice if it appears in both shelves.

```
--Jiashuo Wang(364341)-DBSAssignment1
```

```
SET SCHEMA 'goodreads';
```

```
--A1
```

```
select book_id, title, year_published as pub_year  
FROM book;
```

```
--A2
```

```
select distinct shelf  
from book;
```

```
--B1
```

```
select *  
FROM book  
WHERE date_finished between '2020-1-1' and '2020-12-31';
```

```
--B2
```

```
select book.title  
from book  
WHERE title LIKE 'The%' or title LIKE '_a%';
```

```
--B3
```

```
select *  
FROM book  
WHERE isbn IS NULL;
```

```
--C1
```

```
select COUNT(*) AS total_books  
FROM book;
```

```
--C2
```

```
select MIN(page_count) AS min_pages,  
       MAX(page_count) AS max_pages,  
       AVG(page_count) AS avg_pages  
FROM book;
```

--D1

```
select book.shelf,  
       count(*) as Books_Per_Shelf,  
       avg(avg_rating) as Avg_Ranting  
from book  
group by book.shelf;
```

--D2

```
select author_id,  
       avg(avg_rating) as Avg_rating1  
from book  
group by author_id  
HAVING avg(avg_rating) >4.2;
```

--E1

```
select *  
from book  
where page_count IS NOT NULL  
order by page_count DESC  
LIMIT 10;
```

--E2

```
select book.title,  
       book.year_published  
from book  
ORDER BY title ASC , year_published DESC ;
```

--F1

```
select book_id,  
       title,  
       CONCAT_WS(' ', a.first_name, a.middle_name, a.last_name) AS author_full_name  
FROM book b JOIN author a  
       ON b.author_id = a.author_id;
```

```
--F2
select a.author_id,
       CONCAT_WS(' ', a.first_name, a.middle_name, a.last_name) AS
author_full_name,
       count(b.book_id) as Number_Of_Books
from book b LEFT JOIN author a
              on a.author_id = b.author_id
group by a.first_name, a.middle_name, a.last_name, a.author_id
ORDER BY count(b.book_id) DESC NULLS LAST ;
```

```
--G
--UNION
select title
from book
WHERE shelf = 'read'
UNION
SELECT title
FROM book
WHERE shelf = 'to-read';
```

```
--UNION ALL
select title
FROM book
WHERE shelf = 'read'
UNION ALL
SELECT title
FROM book
WHERE shelf = 'to-read';
```